

單元 1：從 App Engine webapp2 遷移至 Flask

來源：<https://codelabs.developers.google.com/codelabs/cloud-gae-python-migrate-1-flask?hl=zh-tw>

步驟數：8

瞭解如何將 Python App Engine 應用程式從 Web2 遷移至 Flask 網路架構。

目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 新增 Flask 第三方程式庫](#)
5. [5. 更新應用程式檔案](#)
6. [6. 更新 HTML 範本檔案](#)
7. [7. 摘要/清除](#)
8. [8. 其他資源](#)

1. 總覽

本系列程式碼研究室 (可自行調整步調的實作教學課程) 旨在協助 [Google App Engine](#) (標準) 開發人員完成一系列遷移作業，進而將應用程式現代化。最重要的步驟是捨棄原始的執行階段隨附服務，因為[新一代執行階段](#)更具彈性，可為使用者提供更多服務選項。改用新一代執行階段後，您就能更輕鬆地與 Google Cloud 產品整合、使用更多支援的服務，以及支援目前的語言版本。

本教學課程將說明現代化 App Engine 應用程式網路架構的第一個遷移步驟：從 [webapp2](#) 遷移至 [Flask](#)。您可以在應用程式中使用任何處理路徑的網路架構，但本教學課程會使用 Flask，因為這個架構廣受社群採用。

注意：所有含有 `webapp2` 程式碼的後端應用程式都應執行這項遷移作業，包括沒有面向使用者網頁 UI 的應用程式。

課程內容

- 使用第三方程式庫 (內建或其他)
- 更新設定檔
- 將簡易應用程式從 `webapp2` 遷移至 Flask

軟硬體需求

- 具有下列項目的 [Google Cloud Platform 專案](#)：
 - [有效的 GCP 帳單帳戶](#)
 - [已啟用 App Engine](#)
- Python 基礎技能
- 具備基本 Linux 指令的實務知識
- 具備[開發](#)及[部署](#) App Engine 應用程式的基本知識

問卷調查

2. 背景

2008 年，App Engine 首次[在 Python 2.5 上推出](#)時，`webapp` 架構已隨附在內。2013 年，[2.7 執行階段淘汰 2.5 版](#)後，`webapp2` 繼任者在幾年後取代了這個版本。

`webapp2` (請參閱[文件](#)) 仍存在，且可在 App Engine 以外做為符合 WSGI 的網頁架構使用，但不會將使用者要求自行路由至應用程式中的適當程式碼。而是依賴 App Engine、設定檔和開發人員，將網路流量路由至對應的「處理常式」。此外，`webapp2` 的核心優勢與 App Engine 的套裝服務密不可分，因此即使 `webapp2` [適用於 Python 3](#)，也等同於已淘汰 (另請參閱[相關問題](#))。

本單元提供實務練習，讓您將簡單的 `webapp2` 應用程式遷移至 Flask。Flask 是 App Engine 支援的架構，Google Cloud 以外的許多服務也支援 Flask，因此應用程式的可攜性更高。如果不想將應用程式遷移至 Flask，只要該架構有自己的路由功能，您也可以選取其他架構。本程式碼研究室會向資訊科技決策者 (ITDM) 和開發人員說明遷移步驟，無論您實際遷移至哪個架構，都能熟悉這個程序。

這項遷移作業的主要步驟如下：

1. 設定/準備工作
 2. 新增 Flask 第三方案式庫
 3. 更新應用程式檔案
 4. 更新 HTML 範本檔案
-

3. 設定/準備工作

在開始進行本教學課程的主要部分之前，請先設定專案、取得程式碼，然後 (重新) 熟悉 `gcloud` 指令，並部署基準應用程式，確保我們從可運作的程式碼開始。

1. 設定專案

如果您是現有開發人員，[App Engine 資訊主頁](#)可能已顯示您正在執行的服務。在本教學課程中，建議您[建立全新專案](#)，或重複使用現有專案。確認專案已啟用有效的帳單帳戶和 App Engine (應用程式)。

2. 下載基準範例應用程式

GAE [遷移存放區](#)包含您所需的所有程式碼。複製或下載 ZIP 檔案。在本教學課程中，您會從「Module 0」資料夾 (START) 中的程式碼開始，完成教學課程後，您的程式碼應與「Module 1」資料夾 (FINISH) 中的程式碼相符。如果不是，請查看兩者之間的差異，然後繼續下一個實驗室。

- 開始：[模組 0 程式碼](#)
- 完成：[第 1 單元程式碼](#)
- [整個存放區](#) (複製或下載 ZIP 檔案)

Module 0 資料夾應包含如下所示的檔案，如 POSIX `ls` 指令所示：

```
$ ls
app.yaml      index.html    main.py
```

3. (重新) 熟悉 `gcloud` 指令

如果您的電腦上還沒有 `gcloud` 指令，請安裝 [Google Cloud SDK](#)，並確保 `gcloud` 可做為執行路徑的一部分，同時熟悉下列 `gcloud` 指令：

1. `gcloud components update`：更新 Google Cloud SDK
2. `gcloud auth login` - 登入已通過驗證的帳戶
3. `gcloud config list` - 列出 GCP 專案設定
4. `gcloud config set project PROJECT_ID`：設定 GCP 專案 ID
5. `gcloud app deploy` - 部署 App Engine 應用程式

如果您最近沒有使用 `gcloud` 開發 App Engine，請先執行前四個指令 (第 1 到 4 個)，完成設定後再繼續下一個步驟。讓我們快速瀏覽這些指令。

首先，`gcloud components update` 可確保您擁有最新版 Cloud SDK。執行這項指令後，應該會輸出如下內容：

```
$ gcloud components update
```

```
Your current Cloud SDK version is: 317.0.0
```

```
You will be upgraded to version: 318.0.0
```

These components will be updated.			
Name	Version	Size	
Cloud SDK Core Libraries	2020.11.06	15.5 MiB	
gcloud cli dependencies	2020.11.06	10.6 MiB	

The following release notes are new in this upgrade.

Please read carefully for information about new features, breaking changes, and bugs fixed. The latest full release notes can be viewed at:

https://cloud.google.com/sdk/release_notes

318.0.0 (2020-11-10)

. . .

(release notes)

. . .

Subscribe to these release notes at

<https://groups.google.com/forum/#!forum/google-cloud-sdk-announce>.

Do you want to continue (Y/n)?

Creating update staging area	
Uninstalling: Cloud SDK Core Libraries	
Uninstalling: gcloud cli dependencies	
Installing: Cloud SDK Core Libraries	

```
|— Installing: gcloud cli dependencies —|
```

```
|— Creating backup and activating new installation —|
```

Performing post processing steps...done.

Update done!

To revert your SDK to the previously installed version, you may run:

```
$ gcloud components update --version 317.0.0
```

接著，請使用 `gcloud auth login` 驗證身分，以便日後發出 `gcloud` 指令：

```
$ gcloud auth login
```

Your browser has been opened to visit:

```
https://accounts.google.com/o/oauth2/auth?response_type=code&client_id= . . .
```

You are now logged in as [YOUR_EMAIL].

Your current project is [PROJECT_ID]. You can change this setting by running:

```
$ gcloud config set project PROJECT_ID
```

使用 `gcloud config list` 查看目前的專案設定：

```
$ gcloud config list
```

```
[core]
```

```
account = YOUR_EMAIL
```

```
disable_usage_reporting = False
```

```
project = PROJECT_ID
```

```
Your active configuration is: [default]
```

上述指令應會引導您建立新專案或選取現有專案。如果 `gcloud config list` 的輸出內容與您要在本教學課程中使用的專案不符，請執行 `gcloud config set project PROJECT_ID` 來設定專案 ID。接著，再次執行 `gcloud config list`，確認已設定正確的專案 ID。

```
$ gcloud config set project PROJECT_ID
```

```
Updated property [core/project].
```

如果您偏好使用 [Cloud 控制台](#)，可以按照使用者介面建立新專案 (如有需要)，或使用現有的專案。在專案資訊主頁上，您應該會看到專案資訊卡，其中顯示專案 ID (以及專案名稱和編號)：



專案資訊卡

最後一個指令 (#5) `gcloud app deploy` 是用來將應用程式部署至 App Engine。由於我們才剛開始，現在執行是選用步驟，但我們當然不反對部署第 0 模組程式碼，確認程式碼是否正常運作。執行時，請選取要讓應用程式執行的地理區域 (通常是您所在的位置)。但設定後就無法更改。然後查看其餘部署資訊。完成後，系統會通知您應用程式的服務網址。以下是您可能會看到的部分內容：

```
$ gcloud app deploy
Services to deploy:

descriptor:      [/private/tmp/mod0-baseline/app.yaml]
source:          [/private/tmp/mod0-baseline]
target project:  [PROJECT_ID]
target service:  [default]
target version:  [20201116t220827]
target url:      [https://PROJECT_ID.REG_ABBR.r.appspot.com]

Do you want to continue (Y/n)?

Beginning deployment of service [default]...

┌───────────────────────────────────────────────────────────────────────────────────┐
│ ─ Uploading 1 file to Google Cloud Storage                                     ─ │
└───────────────────────────────────────────────────────────────────────────────────┘

File upload done.
Updating service [default]...done.
Setting traffic split for service [default]...done.
Deployed service [default] to [https://PROJECT_ID.REG_ABBR.r.appspot.com]

You can stream logs from the command line by running:
    $ gcloud app logs tail -s default

To view your application in the web browser run:
    $ gcloud app browse
```

如果您有一段時間未使用 App Engine，可能會發現原始部署 `appcfg.py update` 指令已取代為 `gcloud app deploy`。如要進一步瞭解 `gcloud app deploy`，請參閱[說明文件頁面](#)。

最近的另一項變更，是將已部署應用程式的網址從 `http://PROJECT_ID.appspot.com` 調整為 `http://PROJECT_ID.REG_ABBR.r.appspot.com`。大多數應用程式最終都會轉換為新格式。如要進一步瞭解網址格式，請參閱[要求和轉送說明文件](#)。

部署應用程式後，請重新整理瀏覽器(可能需要幾次)，即可查看最新造訪次數：

VisitMe example

Last 10 visits

- Tue Nov 17 06:11:51 2020 from [REDACTED]: Mozilla/5.0 (X11; CrOS x86_64 13421.89.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.183 Safari/537.36
- Tue Nov 17 06:09:19 2020 from [REDACTED]: curl/7.64.1
- Thu Nov 12 06:07:54 2020 from [REDACTED]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.193 Safari/537.36
- Thu Nov 12 06:04:15 2020 from [REDACTED]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.193 Safari/537.36
- Thu Nov 12 06:03:25 2020 from [REDACTED]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.193 Safari/537.36
- Thu Nov 12 02:52:33 2020 from [REDACTED]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.193 Safari/537.36
- Thu Nov 12 02:45:54 2020 from [REDACTED]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.111 Safari/537.36
- Thu Nov 12 02:39:13 2020 from [REDACTED]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.111 Safari/537.36
- Thu Nov 12 02:37:37 2020 from [REDACTED]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.111 Safari/537.36
- Sat Oct 31 08:04:23 2020 from [REDACTED]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.111 Safari/537.36

如果應用程式是新上架，您只會看到一或幾次造訪。

4. 新增 Flask 第三方程式庫

Python 2 App Engine 執行階段提供一組「[內建](#)」[第三方程式庫](#)，您只需在 `app.yaml` 檔案中指定這些程式庫即可使用。雖然這次遷移作業不需要使用這些函式，但下一個遷移教學課程 (第 2 模組) 會用到。

非內建的第三方程式庫必須在名為 `requirements.txt` 的檔案中指定，並在本機安裝在 `lib` 資料夾中，該資料夾與應用程式程式碼位於相同目錄，所有內容都會上傳至 App Engine。如需更多資訊，請參閱[第三方程式庫的組合說明文件](#)。

對於 Flask 等複製的程式庫，您必須使用 `appengine_config.py` 設定檔，告知 App Engine 在 `lib` 資料夾中尋找這些程式庫。`appengine_config.py` 設定檔會放在與 `requirements.txt` 和 `lib` 相同的頂層應用程式資料夾中。在本教學課程的這一部分，您將：

- 建立 `requirements.txt` (指定複製的 [非內建] 第三方程式庫)
- 建立 `appengine_config.py` (辨識第三方程式庫)
- 安裝 (第三方) 套件和依附元件

1. 建立「`requirements.txt`」

建立 `requirements.txt` 檔案來指定套件。在本例中，Flask 是必要的第三方程式庫。撰寫本文時，最新版本為 1.1.2，因此請建立 `requirements.txt`，並加入以下這行程式碼：

```
Flask==1.1.2
```

若要進一步瞭解可接受的格式，請參閱 [requirements.txt 說明文件](#)。

注意：GitHub 存放區中的程式碼可能比本教學課程更為新穎。舉例來說，`requirements.txt` 檔案的 Flask 穩定版本可能比上述使用的版本更新。

2. 建立「`appengine_config.py`」

下一步是讓 App Engine 辨識外部第三方程式庫。建立名為 `appengine_config.py` 的檔案，並在當中加入下列內容：

```
from google.appengine.ext import vendor

# Set PATH to your libraries folder.
PATH = 'lib'
# Add libraries installed in the PATH folder.
vendor.add(PATH)
```

這段程式碼的作用與我們稍早指定的一模一樣，也就是將 App Engine 指向複製的程式庫所在的 `lib` 資料夾。

3. 安裝套件和依附元件

現在請執行 `pip install` 指令建立 `lib` 資料夾，並在其中安裝 Flask 和相關依附元件：

```
$ pip install -t lib -r requirements.txt
```

警告：許多開發人員的系統都安裝了 Python 2 和 3，由於這些教學課程是為 Python 2 開發人員設計，因此最好使用 `pip2` 指令，明確為 Python 2 執行 `pip` 指令。舉例來說，您可以執行 `pip2 install -t lib -r requirements.txt`，取代上述做法。

無論您使用 `pip` 或 `pip2`，套件安裝完成後，應該會有 `lib` 資料夾，內容類似於：

```
$ ls lib
bin/
click/
click-7.1.2.dist-info/
flask/
Flask-1.1.2.dist-info/
itsdangerous/
itsdangerous-1.1.0.dist-info/
jinja2/
Jinja2-2.11.2.dist-info/
markupsafe/
MarkupSafe-1.1.1.dist-info/
werkzeug/
Werkzeug-1.0.1.dist-info/
```

步驟 5 · 約 8 分鐘

5. 更新應用程式檔案

現在更新應用程式檔案 `main.py`。

1. 匯入

如同所有 Python 檔案，匯入作業會優先執行。 `webapp2` 架構匯入作業後，接著是 `ndb` Datastore 程式庫，最後是處理 Django 風格範本的 App Engine 擴充功能。畫面應顯示如下：

- 更新前：

```
import webapp2
from google.appengine.ext import ndb
from google.appengine.ext.webapp import template
```

改用 Flask 時，您會同時匯入 Flask 和範本算繪器。刪除與 `webapp2` 相關的匯入項目，並取代為下列項目 (保留 `ndb` 匯入項目)：

- AFTER：

```
from flask import Flask, render_template, request
from google.appengine.ext import ndb
```

2. 啟動

使用 `webapp2` 的應用程式需要單一陣列 (Python 清單)，列出任何 Python 檔案中的所有路徑和處理常式 (可能還有其他檔案)：

- 更新前：

```
app = webapp2.WSGIApplication([
    ('/', MainHandler),
], debug=True)
```

請注意， `app.yaml` 會執行較高層級的路由作業，並可能呼叫不同的處理常式。這個範例應用程式非常簡單，所有路徑都會連往 `main.py` 處理常式。

Flask 不會使用這類路由表，因此請刪除 `main.py` 中的這些行。Flask 也需要初始化，因此請在 `main.py` 的頂端，匯入項目正下方新增下列程式碼：

- AFTER：

```
app = Flask(__name__)
```

在 Flask 中，您需要先初始化架構，然後使用裝飾器定義路徑。此外，路徑會與函式配對，而非類別或方法。

本程式碼研究室不會納入 Flask 教學課程，因此請花點時間完成 [Flask 教學課程](#)，並詳閱 [Flask 說明文件](#)，進一步熟悉這個架構。

3. 資料模型

這裡沒有任何變更。下一個程式碼研究室將著重於 Datastore。

4. 處理常式

無論您使用哪一個架構 (webapp2 或 Flask)，應用程式都會執行 3 項操作：

1. 處理根路徑 (/) GET 要求
2. 註冊網頁「造訪」(建立/儲存 visit 物件)
3. 顯示最近 10 次造訪的記錄 (使用預先定義的範本 index.html)

webapp2 架構採用以類別為基礎的執行模型，會為每個支援的 HTTP 方法建立處理常式。在我們的簡單案例中，只有 GET，因此定義了 get() 方法：

- 更新前：

```
class MainHandler(webapp2.RequestHandler):
    def get(self):
        store_visit(self.request.remote_addr, self.request.user_agent)
        visits = fetch_visits(10) or () # empty sequence if None
        tpl = os.path.join(os.path.dirname(__file__), 'index.html')
        self.response.out.write(template.render(tpl, {'visits': visits}))
```

如上所述，Flask 會自行進行路由。您不必使用處理常式類別，而是編寫函式，並使用應呼叫函式的路徑加以裝飾。使用者可以在裝飾器呼叫中指定處理的 HTTP 方法，也就是 @app.route('/app/', methods=['GET', 'POST'])。由於預設值只有 GET (和隱含的 HEAD)，因此可以省略。

遷移至 Flask 時，請將 MainHandler 類別及其 get() 方法替換為下列 Flask 路由函式：

- AFTER：

```
@app.route('/')
def root():
    store_visit(request.remote_addr, request.user_agent)
    visits = fetch_visits(10) or () # empty sequence if None
    return render_template('index.html', visits=visits)
```

當然，這並非代表您的應用程式，因為您的應用程式一定比這個範例複雜。這些教學課程的主要目標之一，是協助您開始使用、建立一些「肌肉記憶」，並瞭解要在 App Engine 專屬程式碼中進行哪些變更。如要確認您已正確完成這項變更，請與第 1 堂課 main.py 比較。

5. 輔助檔案

.gcloudignore 檔案沒有任何變更。其用途是指定不要部署到 App Engine 的檔案，這些檔案並非部署及執行應用程式的必要項目，包括但不限於輔助 Python、來源控管、存放區樣板和其他檔案。我們的 .gcloudignore 如下所示 (為求簡潔，已移除註解)：

```
.gcloudignore  
.git  
.gitignore  
.hgignore  
.hg/  
*.pyc  
*.pyo  
__pycache__/  
/setup.cfg  
README.md
```

6. 更新 HTML 範本檔案

1. 移動範本檔案

在基準存放區資料夾 (模組 0) 中，`index.html` 範本檔案與應用程式檔案位於相同資料夾。由於 Flask 需要將 HTML 檔案放在 `templates` 資料夾中，因此您必須建立該資料夾 (`mkdir templates`)，然後將 `index.html` 移至該資料夾。在 Linux 或 Mac OS X 等符合 POSIX 規範的系統中，指令如下：

```
mkdir templates
mv index.html templates
```

2. 更新範本檔案

將 `index.html` 移至 `templates` 後，請進行一項小幅但必要的編輯。讓我們完整查看原始範本檔案：

```
<!doctype html>
<html>
<head>
<title>VisitMe Example</title>
<body>

<h1>VisitMe example</h1>
<h3>Last 10 visits</h3>
<ul>
{% for visit in visits %}
    <li>{{ visit.timestamp.ctime }} from {{ visit.visitor }}</li>
{% endfor %}
</ul>

</body>
</html>
```

`webapp2` 使用 Django 範本，執行可呼叫的項目 (例如 `visit.timestamp.ctime` 沒有括號 `()`)，但 Jinja2 則需要明確指定。雖然這聽起來只是小幅調整，但 Jinja 範本內建的功能更強大，因為您可以在呼叫中傳遞引數。

在 Django 中，您必須建立「範本標記」或撰寫篩選器。瞭解這點後，請在 `visit.timestamp.ctime` 呼叫中新增一組括號，更新 `index.html`：

- 更新前：

```
<li>{{ visit.timestamp.ctime }} from {{ visit.visitor }}</li>
```

- AFTER：

```
<li>{{ visit.timestamp.ctime() }} from {{ visit.visitor }}</li>
```

這是唯一需要進行的變更，其餘遷移程式碼研究室 `index.html` 不需要進行其他變更。

7. 摘要/清除

部署應用程式

完成本教學課程中的所有變更後，應用程式資料夾中的檔案應與[第 1 模組存放區資料夾](#)中的檔案相同 (或幾乎相同)。現在請部署並確認 Module 1 Flask 應用程式的執行方式與 Module 0 `webapp2` 版本相同。

請使用 `gcloud app deploy` 指令，就像先前部署原始的 Module 0 程式碼一樣。透過網頁瀏覽器或 `curl` 或 `wget` 指令存取 `PROJECT_ID.appspot.com` 的應用程式，確認應用程式是否正常運作。

如果發生伺服器錯誤，通常表示 Python 程式碼中有錯字。請查看[應用程式記錄檔](#)進行調查。此外，也請將您的檔案與第 1 模組存放區中的檔案進行比較 (連結就在上方)。

選用：清除

在準備進行下一個遷移程式碼研究室之前，請先清除資源，以免產生帳單費用。身為現有開發人員，您可能已熟悉 [App Engine](#) 的價格資訊。

重要事項：原始 App Engine 執行階段不需要有效的帳單帳戶，但[現在已不再是如此](#) (2019 年 11 月開始，2020 年 4 月開始強制執行)。不過請放心，App Engine 仍提供[免費方案](#)，只要不超出限制，就不會產生任何費用。

選用：停用應用程式

如果還沒準備好進行下一個教學課程，請[停用應用程式](#)，以免產生費用。準備好進行下一個程式碼研究室時，可以重新啟用這項功能。應用程式停用後，不會產生任何流量，因此不會產生費用。不過，如果[資料儲存空間用量](#)超出[免費配額](#)，您仍須支付相關費用，因此請刪除足夠的資料，確保用量低於上限。

另一方面，如果您不打算繼續遷移，並想完全刪除所有內容，可以[關閉專案](#)。

後續步驟

有兩個遷移模組會以完成的模組 1 程式碼為基礎，分別是模組 2 和 7：

- [第 2 個模組](#) (使用 Datastore 時為必填)
 - 從 App Engine `ndb` 遷移至 Cloud NDB
 - 改用 Cloud NDB 後，您可以使用許多其他選項
 - 將應用程式容器化，以便在 Cloud Run 上執行
 - 將應用程式進一步遷移至 Cloud Datastore 用戶端程式庫
 - 將應用程式遷移至 Cloud Firestore，存取 Firebase 功能
- [模組 7](#) (如果您使用 [推送] 工作佇列，則為必填)
 - 新增 App Engine (推送) `taskqueue` 用量
 - 準備將第 1 堂的應用程式遷移至第 8 堂的 [Cloud Tasks](#)

8. 其他資源

App Engine 遷移模組程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- [搜尋問題](#)
- [回報新問題/提供意見](#)

遷移資源

下表提供模組 0 (START) 和模組 1 (FINISH) 的存放區資料夾連結。您也可以從[所有 App Engine 遷移作業的存放區](#)存取這些範例，並複製或下載 ZIP 檔案。

Codelab	Python 2	Python 3
單元 0	code	(不適用)
Module 1	code	(不適用)

App Engine 資源

以下是與這項特定遷移作業相關的其他資源：

- Python 微型網路架構
 - [webapp2 repo](#)
 - [webapp2 文件](#)
 - [Flask 存放區](#)
 - [Flask 說明文件](#)
- (舊版) 從 Python 2.5 遷移至 2.7，並從 `webapp` 遷移至 `webapp2`
 - [Python 2.5 已於 2013 年淘汰，並於 2017 年停止支援，改用 2.7 版](#)
 - [Python 2.5 至 2.7 文件](#)
 - [webapp 到 webapp2 文件](#)
- 遷移至 Python 3 和 GAE 次世代執行階段
 - [遷移作業首頁](#)
- 一般
 - [在 Google Cloud Platform 上執行 Python](#)
 - [Google Cloud Python 用戶端程式庫](#)